

CSP 算法模板 • 日常学习风格版

C++17 / A4 打印资料。根据你的“日常学习”代码改写：普通函数、vector、引用参数；保留熟悉的命名，循环与判断分行写。不使用 lambda、泛型函数或额外的图类型别名。long long 直接写全。

快速选择

连通块、可达性 → 01 DFS

枚举排列、选择与撤销 → 02 回溯

等权最少步数、同时扩散 → 03-04 BFS

非负边权最短路 → 05 Dijkstra

先后依赖、DAG 最长路 → 06 拓扑排序

合并集合、判断连通 → 07 并查集

最小代价连接所有点 → 08 Kruskal

最大化最小值、单调判定 → 09 二分答案

单点加、区间求和 → 10 树状数组

右侧第一个更大、窗口极值 → 11-12 单调栈 / 队列

选物品、限制容量 → 13 背包

最长上升子序列 → 14 LIS

子串比较、精确匹配 → 15 哈希 / 16 KMP

大指数取模、质数模逆元 → 17 快速幂

区间加、区间求和 → 附录 C 线段树

使用约定

每节独立选用，复制所需函数到 main 前面；不要把所有章节直接混在一起，同名 dfs、init 等可能冲突。只有第 08 节需要一起复制第 07 节并查集。图通常是 1 下标，网格和字符串是 0 下标，各节会注明。

下面是公共头。INF 表示不可达，不参与正常答案运算；求和和乘法的中间结果也必须不溢出。附录 B、C 则是可以独立编译的完整程序。

```
#include<iostream>
#include<vector>
#include<queue>
#include<stack>
#include<deque>
#include<algorithm>
#include<string>
#include<functional>
#include<utility>
using namespace std;
const long long INF = (1LL << 62);
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    // 读入，初始化，调用函数，输出
    return 0;
}
```

01 DFS：图的遍历

来源：日常学习/装箱问题02.cpp 的递归组织方式；图遍历为补充

图下标 $1..n$ ， $O(n+m)$ 。 vis 表示这个点是否访问过，图遍历结束后不撤销。无向图统计连通块时，从每个没访问过的点开始 dfs。链状图很深时递归可能爆栈，改用下面的栈版本。

```
void dfs(int u, vector<vector<int>> &g, vector<bool> &vis) {
    vis[u] = true;
    // 在这里处理当前点 u
    for(int v : g[u]) {
        if(!vis[v]) {
            dfs(v, g, vis);
        }
    }
}

// 调用: vector<bool> vis(n + 1, false); dfs(s, g, vis);
// 求无向图连通块: 枚举 i=1..n, 没访问过就 cnt++, dfs(i, g, vis)。
```

// 点很多、递归可能太深时，用栈代替递归

```
void dfs_stack(int s, vector<vector<int>> &g, vector<bool> &vis) {
    stack<int> st;
    st.push(s);
    vis[s] = true;
    while(!st.empty()) {
        int u = st.top();
        st.pop();
        for(int v : g[u]) {
            if(!vis[v]) {
                vis[v] = true;
                st.push(v);
            }
        }
    }
}
```

02 DFS：回溯枚举

来源：日常学习/backward_digital_sum.cpp；日常学习/cow_in_skycraper.cpp

沿用你的 memo、vis、dfs(idx,n) 写法，下面枚举 1..n 的排列。vis=true 表示已经使用。先选择、递归、再撤销； $O(n \cdot n!)$ （含输出）。与图遍历不同，回溯必须恢复本层修改。

```
vector<int> memo;
vector<bool> vis;
void dfs(int idx,int n){
    if(idx == n + 1){
        for(int i = 1;i <= n;i++){
            cout << memo[i] << ' ';
        }
        cout << '\n';
        return;
    }
    for(int i = 1;i <= n;i++){
        if(vis[i]){
            continue;
        }
        memo[idx] = i;
        vis[i] = true;
        dfs(idx + 1,n);
        vis[i] = false; // 撤销选择，让其他分支也能用 i
    }
}
// main 里先初始化，再开始搜索：
// memo.assign(n + 1,0);
// vis.assign(n + 1,false);
// dfs(1,n);
// 找一个解就结束：改为 bool dfs，找到后 return true，参考你的原题。
```

03 BFS：无权图最短路

来源：参考日常学习/最长路BFS.cpp 的队列写法；改为无权最短路

边权相同才可用普通 BFS 求最少步数； $O(n+m)$ 。dist=-1 表示没访问过或不可达。入队时就更新距离，不能等到出队再标记。

```
void bfs(int s,vector<vector<int>> &g,vector<int> &dist){
    dist.assign(g.size(),-1);
    queue<int> q;
    dist[s] = 0;
    q.push(s);
    while(!q.empty()){
        int u = q.front();
        q.pop();
        for(int v : g[u]){
            if(dist[v] == -1){
                dist[v] = dist[u] + 1;
                q.push(v);
            }
        }
    }
}
// vector<int> dist;
// bfs(s,g,dist);
// cout << dist[t]; // 这里约定不可达输出 -1，实际看题目要求
```

04 BFS：网格与多源扩散

来源：暑期训练营代码/Day2/血色先锋队.cpp；日常学习/SEARCH.cpp

grid 是矩形网格，0 下标，X 是障碍。src 中源点坐标必须有效；多源一起入队，单源传 $\{\{sx, sy\}\}$ 。 $O(n*m+源点数)$ 。注意行数 n 和列数 m 不一样。

```
void bfs(vector<string> &grid, vector<pair<int, int>> &src,
        vector<vector<int>> &dist) {
    int n = grid.size();
    int m = 0;
    if (n > 0) {
        m = grid[0].size();
    }
    dist.assign(n, vector<int>(m, -1));
    queue<pair<int, int>> q;
    for (auto [x, y] : src) {
        if (grid[x][y] != 'X' && dist[x][y] == -1) {
            dist[x][y] = 0;
            q.push({x, y});
        }
    }
    int dx[4] = {1, -1, 0, 0};
    int dy[4] = {0, 0, 1, -1};
    while (!q.empty()) {
        auto [x, y] = q.front();
        q.pop();
        for (int i = 0; i < 4; i++) {
            int nx = x + dx[i];
            int ny = y + dy[i];
            if (nx < 0 || nx >= n || ny < 0 || ny >= m) {
                continue;
            }
            if (grid[nx][ny] == 'X' || dist[nx][ny] != -1) {
                continue;
            }
            dist[nx][ny] = dist[x][y] + 1;
            q.push({nx, ny});
        }
    }
}
```

05 Dijkstra: 非负边权最短路

来源: 日常学习/单源最短路径.cpp

保留你原来的 `g`、`dist`、`done`、`pq`。所有边权必须非负; 距离用 `long long`, 所有有效距离须小于 `INF`。堆优化 $O((n+m)\log(n+m))$, 可保留平行边。图 1 下标, 无向边存两次。`INF` 在公共头中定义。

```
void dijkstra(int s, vector<vector<pair<int, long long>>> &g,
              vector<long long> &dist) {
    dist.assign(g.size(), INF);
    vector<bool> done(g.size(), false);
    priority_queue<pair<long long, int>,
                  vector<pair<long long, int>>, greater<>> > pq;

    dist[s] = 0;
    pq.push({0, s}); // 距离在前, 点的编号在后
    while(!pq.empty()) {
        auto [d, u] = pq.top();
        pq.pop();
        if(done[u]) {
            continue;
        }
        done[u] = true;
        for(auto [v, w] : g[u]) {
            if(dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
                pq.push({dist[v], v});
            }
        }
    }
}

// 调用: vector<long long> dist; dijkstra(s, g, dist);
// dist[t] == INF 表示不可达, 按题意输出。
// 边权也须小于 INF, 保证 dist[u]+w 不溢出。
// 记录路径: 更新成功时 pre[v]=u; 从终点回溯, 再 reverse。
```

06 拓扑排序与 DAG 最长路

来源：日常学习/拓扑排序.cpp；日常学习/最长路.cpp

沿用 indeg、ans、topo_q、dp。只用于有向无环图，允许负边； $O(n+m)$ 。拓扑序少于 n 个点说明有环，不能继续套 DAG 最长路。dp=-INF 表示不可达，路径和须在 $(-INF, INF)$ 内。

```
vector<int> topo(vector<vector<pair<int, long long>>> &g, int n) {
    vector<int> indeg(n + 1, 0);
    for(int i = 1; i <= n; i++) {
        for(auto [v, w] : g[i]) {
            indeg[v]++;
        }
    }
    queue<int> q;
    for(int i = 1; i <= n; i++) {
        if(indeg[i] == 0) {
            q.push(i);
        }
    }
    vector<int> ans;
    while(!q.empty()) {
        int u = q.front();
        q.pop();
        ans.push_back(u);
        for(auto [v, w] : g[u]) {
            indeg[v]--;
            if(indeg[v] == 0) {
                q.push(v);
            }
        }
    }
    return ans;
}

// 返回 false 表示有环，dp 通过引用参数带回
bool longest(int s, vector<vector<pair<int, long long>>> &g,
             vector<long long> &dp) {
    int n = (int)g.size() - 1;
    vector<int> topo_q = topo(g, n);
    dp.assign(n + 1, -INF);
    if((int)topo_q.size() != n) {
        return false;
    }
    dp[s] = 0;
    for(int u : topo_q) {
        if(dp[u] == -INF) {
            continue;
        }
        for(auto [v, w] : g[u]) {
            dp[v] = max(dp[v], dp[u] + w);
        }
    }
    return true;
}
```

07 并查集

来源：日常学习/最小生成树.cpp

直接使用你原来的全局 fa、rk 和 init/find/unite，补上同集合判断。路径压缩+按秩合并，均摊 $O(\alpha(n))$ 。
每组数据先 init(n)。

```
vector<int> fa,rk;
void init(int n){
    fa.resize(n + 1);
    rk.assign(n + 1,0);
    for(int i = 0;i <= n;i ++){
        fa[i] = i;
    }
}
int find(int x){
    if(x == fa[x]){
        return x;
    }
    fa[x] = find(fa[x]); // 路径压缩
    return fa[x];
}
void unite(int x,int y){
    x = find(x);
    y = find(y);
    if(x == y){
        return;
    }
    if(rk[x] < rk[y]){
        swap(x,y);
    }
    fa[y] = x;
    if(rk[x] == rk[y]){
        rk[x] ++;
    }
}
// init(n); unite(x,y);
// if(find(x) == find(y)) cout << "同一个集合";
```

08 Kruskal：最小生成树

来源：日常学习/最小生成树.cpp

沿用你的 `edge`、`edges`、`operator<`、`total`、`cnt`。先复制第 07 节并查集。无向图允许负边， $n \geq 1$ ， $O(m \log m)$ 。
`bool` 返回是否连通，`total` 保存答案，避免 `-1` 与合法负权答案冲突。

```
struct edge{
    int u;
    int v;
    long long w;
    bool operator<(const edge &other) const{
        return w < other.w;
    }
};
vector<edge> edges;
bool kruskal(int n, long long &total){
    init(n);
    sort(edges.begin(), edges.end());
    total = 0;
    int cnt = 0;
    for(auto &e : edges){
        if(cnt == n - 1){
            break;
        }
        if(find(e.u) != find(e.v)){
            unite(e.u, e.v);
            total += e.w;
            cnt++;
        }
    }
    return cnt == n - 1;
}
// 每组先 edges.clear(), 读边: edges.push_back({u, v, w});
// long long total;
// if(kruskal(n, total)) cout << total;
// else cout << "orz"; // 不连通的输出按题目要求修改
```


09 二分答案：用跳石头作例子

来源：暑期训练营代码/Day1/跳石头.cpp；沿用普通 ok 函数

不用泛型和 lambda：直接写 `ok(dist,...)`，二分最大可行距离。`location` 是升序内部石头位置，末尾追加终点 `L`。判定为“先 true 后 false”，总复杂度 $O(n \log L)$ 。

```
bool ok(long long dist,vector<long long> &location,int m){
    int move_cnt = 0;
    long long last = 0;
    for(long long x : location){
        if(x - last < dist){
            move_cnt ++;
        }
        else{
            last = x;
        }
    }
    return move_cnt <= m;
}

long long binary_search(long long L,vector<long long> &location,int m){
    long long l = 0;
    long long r = L;
    while(l < r){
        long long mid = l + (r - l + 1) / 2;
        if(ok(mid,location,m)){
            l = mid;
        }
        else{
            r = mid - 1;
        }
    }
    return l;
}

// 调用前 sort(location.begin(),location.end()); location.push_back(L);
// 最后一段过短时，计数相当于移除前一个保留石头，终点不移动。
// 找最小可行值（先 false 后 true），改成：
// mid = l + (r-l)/2;
// if(ok(mid,...)) r=mid; else l=mid+1;
// 两种写法都要求 [l,r] 内至少有一个可行答案。
```

10 树状数组：单点加、区间和

来源：暑期训练营代码/Day5/树状数组1.cpp；改成 vector 全局数组

1 下标，add(0,...) 会死循环。每次操作 $O(\log n)$ 。先设置 n 并 tree.assign(n+1,0)，再逐个 add(i,a[i]) 建树。

```
int n;
vector<long long> tree;
int lowbit(int x){
    return x & -x;
}
void add(int x,long long k){
    while(x <= n){
        tree[x] += k;
        x += lowbit(x);
    }
}
long long sum(int x){
    long long ans = 0;
    while(x > 0){
        ans += tree[x];
        x -= lowbit(x);
    }
    return ans;
}
long long query(int l,int r){
    return sum(r) - sum(l - 1);
}
```

// 区间加、单点查：改为用差分数组建树。

// [l,r]加k: add(l,k); 若r<n, 再add(r+1,-k)。

// 查询第x个值: sum(x)。不要和“单点加、区间和”混用。

11 单调栈：右侧第一个更大元素

来源：暑期训练营代码/Day5/单调栈.cpp

保留你的 data、ans、s，输入 data 为 1 下标，data[0] 不使用。答案是下标，0 表示不存在。 $O(n)$ 。严格更大用 >，大于等于用 >=。

```
vector<int> next_greater(vector<long long> &data){
    int n = (int)data.size() - 1;
    stack<int> s;
    vector<int> ans(n + 1,0);
    for(int i = 1;i <= n;i ++){
        while(!s.empty() && data[i] > data[s.top()]){
            ans[s.top()] = i;
            s.pop();
        }
        s.push(i);
    }
    return ans;
}
```

12 单调队列：滑动窗口最小值

来源：暑期训练营代码/Day5/滑动窗口.cpp

数组 0 下标， $1 \leq k \leq n$ ， $0(n)$ 。q 存下标，ans 存每个窗口的最小值。把 $\geq$ 改成 $\leq$ 就是求最大值。

```
vector<long long> window_min(vector<long long> &a, int k) {
    deque<int> q;
    vector<long long> ans;
    for(int i = 0; i < (int)a.size(); i++) {
        while(!q.empty() && q.front() <= i - k) {
            q.pop_front(); // 移除已经离开窗口的下标
        }
        while(!q.empty() && a[q.back()] >= a[i]) {
            q.pop_back(); // 后来的数更小，前面的大数没有用了
        }
        q.push_back(i);
        if(i >= k - 1) {
            ans.push_back(a[q.front()]);
        }
    }
    return ans;
}
```

13 背包：01 与完全

来源：日常学习/装箱问题.cpp；暑期训练营代码/Day3/疯狂的采药.cpp

拆成两个普通函数，直接对照循环方向。weight、value 都是 0 下标，体积为正。dp[j] 表示容量不超过 j 的最大价值，允许不选。 $O(nV)$ ，空间 $O(V)$ 。恰好装满：只设 $dp[0]=0$ ，其余=-INF，且来源可达才能转移。

```
long long bag01(int V, vector<int> &weight, vector<long long> &value) {
    vector<long long> dp(V + 1, 0);
    for(int i = 0; i < (int)weight.size(); i++) {
        // 每件只能选一次，所以容量倒序
        for(int j = V; j >= weight[i]; j--) {
            dp[j] = max(dp[j], dp[j - weight[i]] + value[i]);
        }
    }
    return dp[V];
}

long long bag_complete(int V, vector<int> &weight,
                       vector<long long> &value) {
    vector<long long> dp(V + 1, 0);
    for(int i = 0; i < (int)weight.size(); i++) {
        // 每件可以选无限次，所以容量正序
        for(int j = weight[i]; j <= V; j++) {
            dp[j] = max(dp[j], dp[j - weight[i]] + value[i]);
        }
    }
    return dp[V];
}

// 装箱最小剩余：让 value[i]=weight[i]，答案 V-bag01(V, weight, value)。
```

14 LIS：最长严格上升子序列

来源：暑期训练营代码/Day3/最长上升子序列.cpp

保留原来的 `tails`、`lower_bound` 写法， $O(n \log n)$ 。严格上升用 `lower_bound`，不下降用 `upper_bound`。
`tails` 的长度是答案，但 `tails` 本身不一定是原数组的一条子序列。

```
int lis(vector<long long> &a){
    vector<long long> tails;
    for(long long x : a){
        auto it = lower_bound(tails.begin(), tails.end(), x);
        if(it == tails.end()){
            tails.push_back(x);
        }
        else{
            *it = x;
        }
    }
    return tails.size();
}
```

15 字符串哈希：子串比较

来源：暑期训练营代码/Day9/字符串匹配.cpp

沿用全局 `hash_num`、`pw` 和 `get` 函数，不用类。`s` 是 0 下标；`get(l, r)` 取半开区间 $[l, r)$ ，要求 $0 \leq l < r \leq s.size()$ 。 $O(n)$ 预处理， $O(1)$ 查询。哈希有碰撞风险，精确匹配可用 KMP。

```
vector<unsigned long long> hash_num, pw;
void init_hash(string &s){
    int n = s.size();
    hash_num.assign(n + 1, 0);
    pw.assign(n + 1, 1);
    for(int i = 0; i < n; i++){
        hash_num[i + 1] = hash_num[i] * 131 + (unsigned char)s[i] + 1;
        pw[i + 1] = pw[i] * 131;
    }
}
unsigned long long get(int l, int r){
    return hash_num[r] - hash_num[l] * pw[r - l];
}
// string s="ababa"; init_hash(s);
// get(0, 3) 和 get(2, 5) 对应两个 "aba"。
// 当前全局数组只保存最近一次 init_hash 的字符串。
```

16 KMP：精确字符串匹配（补充）

来源：补充模板，按普通函数与 vector 写法展开

s 为原串，p 为模式串，均为 0 下标。返回所有匹配起点，允许重叠， $O(n+m)$ 。nxt[i] 是 p[0..i] 最长相等真前后缀长度；j 是当前匹配长度。这里约定空模式返回空结果。

```
vector<int> kmp(string &s, string &p) {
    int m = p.size();
    vector<int> ans;
    if(m == 0) {
        return ans;
    }
    vector<int> nxt(m, 0);
    int j = 0;
    for(int i = 1; i < m; i++) {
        while(j > 0 && p[i] != p[j]) {
            j = nxt[j - 1];
        }
        if(p[i] == p[j]) {
            j++;
        }
        nxt[i] = j;
    }
    j = 0;
    for(int i = 0; i < (int)s.size(); i++) {
        while(j > 0 && s[i] != p[j]) {
            j = nxt[j - 1];
        }
        if(s[i] == p[j]) {
            j++;
        }
        if(j == m) {
            ans.push_back(i - m + 1);
            j = nxt[j - 1]; // 继续找，允许重叠
        }
    }
    return ans;
}
```

17 快速幂与逆元

来源：暑期训练营代码/Day8/模意义下的乘法逆元.cpp；快速幂为补充

$b \geq 0$, $1 \leq \text{mod} \leq 2^{31}-1$, 确保 long long 乘法不会溢出; $O(\log b)$ 。费马逆元要求 p 是质数且 $a \not\equiv 0 \pmod p$, 不适用于任意模数。

```
long long qpow(long long a, long long b, long long mod) {
    a = (a % mod + mod) % mod;
    long long ans = 1 % mod;
    while(b > 0) {
        if(b % 2 == 1) {
            ans = ans * a % mod;
        }
        a = a * a % mod;
        b /= 2;
    }
    return ans;
}
```

// 质数 p 下, a 的逆元: $\text{qpow}(a, p-2, p)$ 。

// 批量求逆元 ($1 \leq n < p$, p 是质数) :

// `vector<long long> inv(n + 1);`

// `inv[1] = 1;`

// `for(int i = 2; i <= n; i++) {`

// `inv[i] = (p - p / i) * inv[p % i] % p;`

// }

附录 A：前缀和、差分与 STL（补充）

```
// 以下是独立片段，变量按题目定义；a 和 pre 均分配 n+1，1 下标。
vector<long long> pre(n+1);
for(int i=1;i<=n;++i) pre[i]=pre[i-1]+a[i];
long long answer=pre[r]-pre[l-1]; // 闭区间 [l,r]

// 差分：d 分配 n+2；初值 d[i]=a[i]-a[i-1]，令 a[0]=0。
d[1]+=v;d[r+1]-=v;
for(int i=1;i<=n;++i) a[i]=a[i-1]+d[i];

// 二维前缀和，数组四周预留一圈 0：
s[i][j]=a[i][j]+s[i-1][j]+s[i][j-1]-s[i-1][j-1];
// [x1..x2] × [y1..y2] 的和：
long long ans=s[x2][y2]-s[x1-1][y2]-s[x2][y1-1]+s[x1-1][y1-1];

sort(a.begin(),a.end());
a.erase(unique(a.begin(),a.end()),a.end()); // 排序去重，也可用于离散化
int pos=lower_bound(a.begin(),a.end(),x)-a.begin(); // 第一个 >=x；可能等于size
// upper_bound：第一个 >x。使用前数组必须升序！
priority_queue<int,vector<int>,greater<int>> pq; // 小根堆
// getline 前若刚使用 cin >>，用 getline(cin >> ws,line) 可跳过前导空白；
// 如果空行/行首空格有意义，则改用 cin.ignore(...) 丢弃上一行剩余内容。
```

原代码提醒与提交检查

- 血色先锋队.cpp: 原列边界误用了行数, 矩形网格会出错; 整理版分别用 `n` 和 `m`。
- 单源最短路径.cpp: 改 `long long` 距离及 `INF`; 不可达输出按题意修改, 不能所有题都输出 2147483647。
- 最长路BFS.cpp: 反复松弛不是普通 BFS; 遇可达正环可能不停更新。整理版采用你“最长路.cpp”的拓扑 DP。
- 字符串匹配.cpp: 原来从 0 调用时访问 `pos-1`; 整理版改 `n+1` 前缀数组。
- 跳石头.cpp: 保留贪心判定, 逐个枚举答案改为二分。
- 装箱问题.cpp: 标准背包按容量定义 `dp`, 不能直接照搬原来按物品数建的状态。
- 最小生成树.cpp: 连接失败与负权答案分开表示。
- 提交前核对: 有向/无向、下标起点、负边、不可达、重边、自环、`n=1`、多组数据初始化。
- 图遍历标记不撤销; 回溯要撤销; BFS 入队时标记; 01 背包倒序, 完全背包正序。
- 这是常用主题整理, 不是全部历史题解的正确性审计。新增模板已注明。

附录 B: Dijkstra 完整程序

输入: $n\ m\ s$, 接着 m 行有向边 $u\ v\ w\ (w \geq 0)$ 。输出源点到各点距离, 不可达输出 -1 ; 按实际题意修改。无向图需加反向边。

```
#include<iostream>
#include<vector>
#include<queue>
#include<functional>
using namespace std;
const long long INF = (1LL << 62);

void dijkstra(int s,vector<vector<pair<int,long long>>> &g,
              vector<long long> &dist){
    dist.assign(g.size(),INF);
    vector<bool> done(g.size(),false);
    priority_queue<pair<long long,int>,
                  vector<pair<long long,int>>,greater<>> > pq;
    dist[s] = 0;
    pq.push({0,s}); // 距离在前, 点的编号在后
    while(!pq.empty()){
        auto [d,u] = pq.top();
        pq.pop();
        if(done[u]){
            continue;
        }
        done[u] = true;
        for(auto [v,w] : g[u]){
            if(dist[u] + w < dist[v]){
                dist[v] = dist[u] + w;
                pq.push({dist[v],v});
            }
        }
    }
}
```

附录 B: Dijkstra 完整程序（续）

```
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, s;
    cin >> n >> m >> s;
    vector<vector<pair<int, long long>>> g(n + 1);
    for(int i = 0; i < m; i++) {
        int u, v;
        long long w;
        cin >> u >> v >> w;
        g[u].push_back({v, w});
    }
    vector<long long> dist;
    dijkstra(s, g, dist);
    for(int i = 1; i <= n; i++) {
        if(dist[i] == INF) {
            cout << -1;
        }
        else {
            cout << dist[i];
        }
        if(i != n) {
            cout << ' ';
        }
    }
    cout << '\n';
    return 0;
}
```

附录 C：线段树完整程序

输入 n m 、 n 个数；1 1 r k ：区间加 k ；2 1 r ：区间求和。1 下标， $n \leq 100000$ 。多组数据须清空 `lazy`。

```
#include<iostream>
#include<vector>
using namespace std;

const int MAXN = 100005;

long long a[MAXN];          // 原始数组
long long tree[MAXN << 2];  // 线段树，区间和
long long lazy[MAXN << 2];  // 懒标记

// 建树
void build(int node, int l, int r){
    if(l == r){
        tree[node] = a[l];
        return;
    }
    int mid = (l + r) >> 1;
    build(node << 1, l, mid);
    build(node << 1 | 1, mid + 1, r);
    tree[node] = tree[node << 1] + tree[node << 1 | 1];
}

// 下传懒标记
void push_down(int node, int l, int r){
    if(lazy[node] == 0) return;
    int mid = (l + r) >> 1;
    int left = node << 1;
    int right = node << 1 | 1;
    // 更新左子节点
    tree[left] += lazy[node] * (mid - l + 1);
    lazy[left] += lazy[node];
    // 更新右子节点
    tree[right] += lazy[node] * (r - mid);
    lazy[right] += lazy[node];
    // 清除当前节点懒标记
    lazy[node] = 0;
}

// 区间更新 [ql, qr] += val
void update(int node, int l, int r, int ql, int qr, long long val){
    if(ql <= l && r <= qr){
        tree[node] += val * (r - l + 1);
        lazy[node] += val;
        return;
    }
    push_down(node, l, r);
    int mid = (l + r) >> 1;
    if(ql <= mid) update(node << 1, l, mid, ql, qr, val);
    if(qr > mid) update(node << 1 | 1, mid + 1, r, ql, qr, val);
    tree[node] = tree[node << 1] + tree[node << 1 | 1];
}
```

附录 C：线段树完整程序（续）

```
// 区间查询 [ql, qr] 的和
long long query(int node, int l, int r, int ql, int qr){
    if(ql <= l && r <= qr){
        return tree[node];
    }
    push_down(node, l, r);
    int mid = (l + r) >> 1;
    long long ans = 0;
    if(ql <= mid) ans += query(node << 1, l, mid, ql, qr);
    if(qr > mid) ans += query(node << 1 | 1, mid + 1, r, ql, qr);
    return ans;
}

int main(){
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;
    cin >> n >> m;
    for(int i = 1; i <= n; i++){
        cin >> a[i];
    }
    build(1, 1, n);

    while(m--){
        int op, x, y;
        cin >> op >> x >> y;
        if(op == 1){
            long long k;
            cin >> k;
            update(1, 1, n, x, y, k);
        } else {
            cout << query(1, 1, n, x, y) << '\n';
        }
    }
    return 0;
}
```